

PRACTICAL CRYPTOGRAPHY

crypt@b-it 2026
Summer school on Cryptography
Sofía Celi



FrodoFIR

What can “practical cryptography” be?

- Not an *easy question*
- Maybe, let's look at the contrary:
 - What is impractical?
 - Communicationally heavy
 - Computationally heavy
 - Financially invalid
 - Difficult to explain
 - Difficult to code
 - Hard to deploy

What can “practical cryptography” be?

So what makes cryptography "practical"?

- Secure enough for the threat model (perhaps not "perfectly" secure)
- Cheap in bandwidth and computation
- Affordable to deploy and operate
- Explainable to engineers and auditors
- Implementable without subtle traps
- Standardized, with mature libraries
- Sound key management and usability

Practical cryptography == security that survives contact with real networks, real hardware, real budgets, and real people.

What can “practical cryptography” be?

So what makes cryptography "practical"?

- All of this, however, can be bypassed if:
 - It is the only available scheme
 - If people agree that it is “worth” it
 - Trust or reputation
- There is no easy answer

What can “practical cryptography” be?

We will look at two things:

- A scheme that was considered impractical and how now it is approaching practicality (with caveats)
- A protocol that is practical, and how it is approaching something less practical and that can have real-world problems (including policy ones)

What can “practical cryptography” be?

Today:

- A scheme that was considered impractical and how now it is approaching practicality (with caveats)
 - **Private Information Retrieval (PIR)**

LECTURE 1: PRIVATE INFORMATION RETRIEVAL:

ARE WE CLOSE TO MAKE IT PRACTICAL?

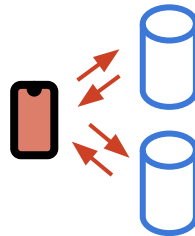


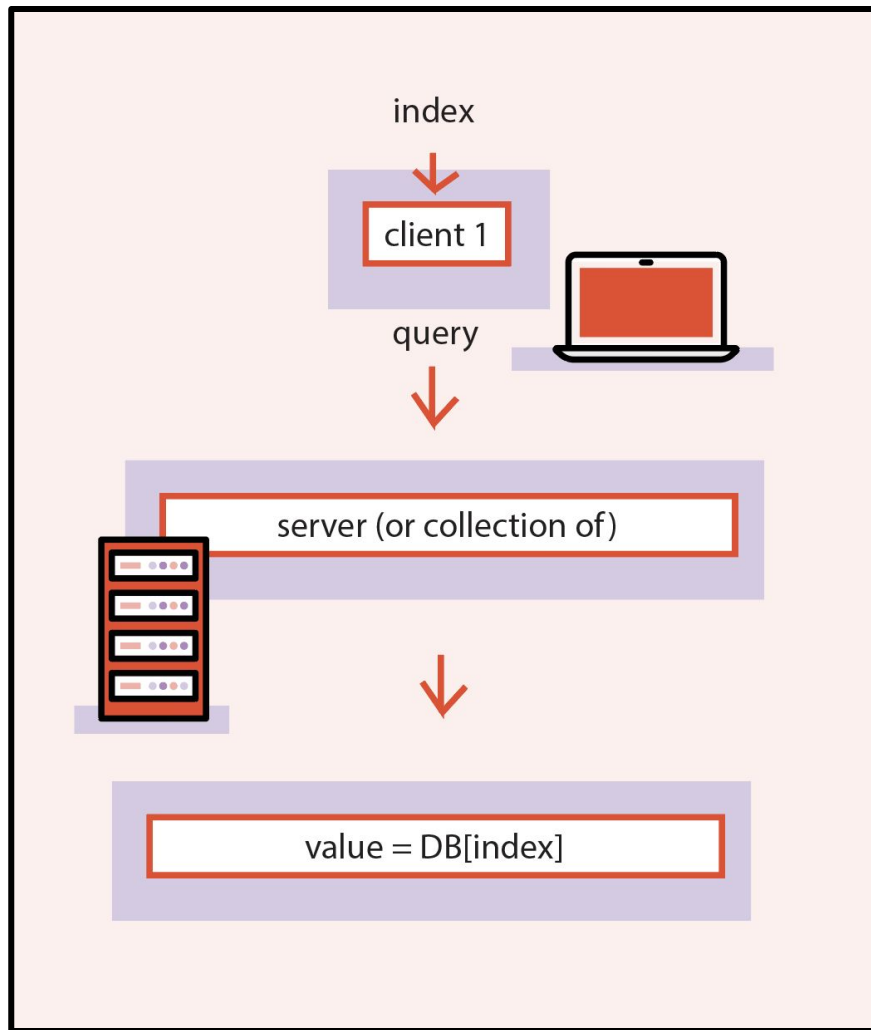
FrodoPIR

Private Information Retrieval (PIR)

A Private Information Retrieval (PIR) scheme provides the ability for clients to **retrieve items from an online public (*) database of m elements**, without **revealing anything about their queries** to the untrusted host server(s)

- Parties:
 - a. Client(s)
 - b. Server (one or multiple)
- Steps:
 - Query
 - Response
 - Parse



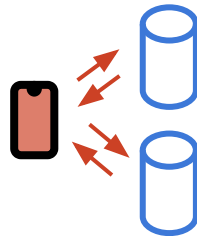


assuming the DB is
public and it is
index by digits

Mise-en-scene

Databases (DBs) are the core of our Internet communications:

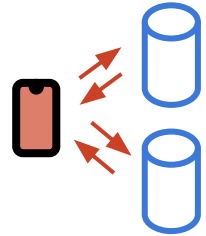
- Almost every system uses it:
 - The Internet and the web are, in fact, a type of DB
- More than *just querying for single exact elements*:
 - Allow for recommendations based on the query
 - Provide means to search the web
 - Provide transparency means: mostly centered on websites/certificates
- This incurs in privacy violations:
 - We reveal our choices
 - We reveal our browsing history
- There is a “practical” reason why we will want privacy here

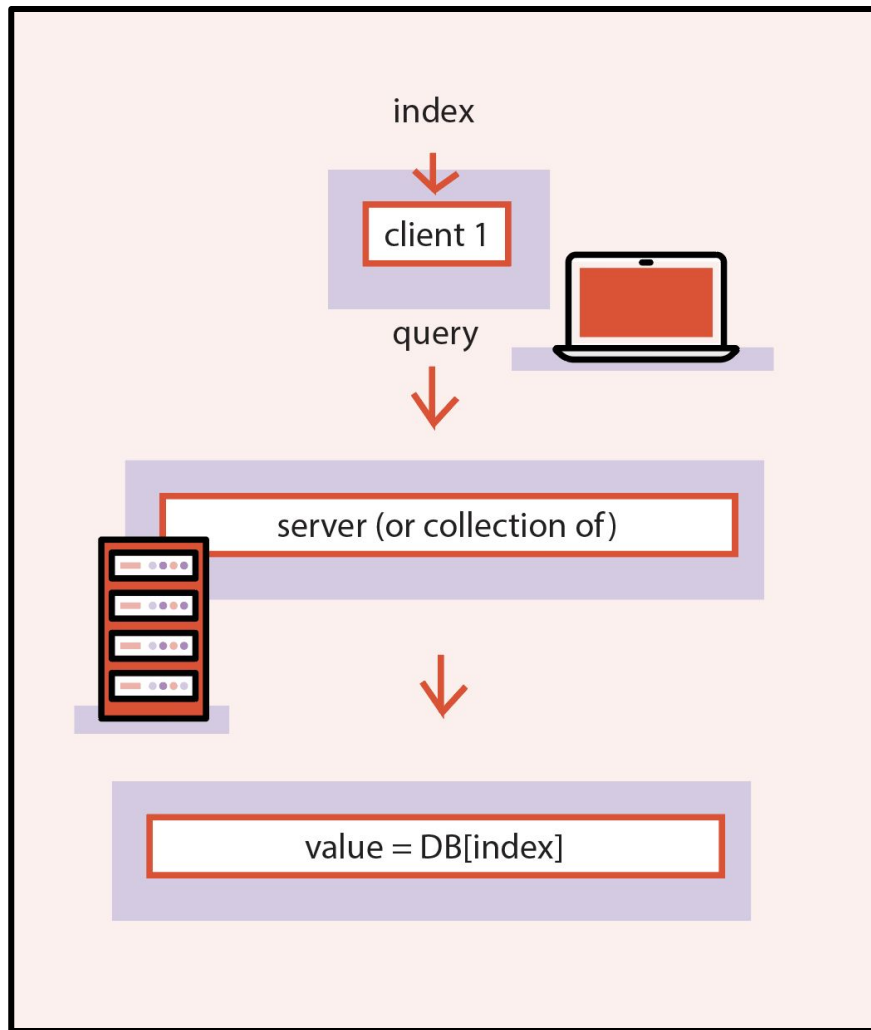


Mise-en-scene

How can we integrate privacy into queries to databases?

- **Start with the simple version of the problem: querying for one element in a indexed-by-digits public DB**
- **Expand to handle batches (AND)**
- **Expand to handle keyword-based DBs**
- Expand to allow for complex operations: joins/disjoins
- Expand for more complex algorithms: obviously navigating a DB to find similar (close) elements, for example
- Expand for private DBs
- Is that all?





assuming the DB is
public and it is
index by digits

Private Information Retrieval (PIR)

Two types (sort of):

1. **Information-theoretic PIR:** client interacting with multiple non-colluding servers
2. **Computational-theoretic PIR:** client interacting with a single server, provides computational security based on cryptographic assumptions:
 - a. Stateless PIR:
 - The client does not store any (pre)information (or *hint*) in order to launch queries
 - The schemes (a bunch!) perform **worse than downloading the whole DB or they require computational overheads**

Private Information Retrieval (PIR)

Two types (sort of):

1. Information-theoretic PIR: client interacting with multiple non-colluding servers
2. **Computational-theoretic PIR**: client interacting with a single server, provides computational security based on cryptographic assumptions:
 - a. Stateless PIR
 - b. **Stateful PIR**: provides a “state” (or hint/digest) used as a “preprocessing” step amortised over n client queries
 - This one, we consider “practical”
 - Sublinear communication (*)
 - Linear computation (*)

Private Information Retrieval (PIR)

Two types (sort of):

1. Information-theoretic PIR: client interacting with multiple non-colluding servers
2. **Computational-theoretic PIR**: client interacting with a single server, provides computational security based on cryptographic assumptions:
 - a. Stateless PIR
 - b. **Stateful PIR**

Idea: *encrypt the query* instead of *secret-sharing* it

Private Information Retrieval (PIR)

Prior limitations in **Computational-theoretic PIR**:

- Expensive pre-processing in terms of computation or communication
- High online-phase bandwidth consumption
- Lack of practical security parameters
- Lack of simple, open-source, available, verified implementations

Current look

- Very active research area:
 - Hint-based (offline/online) PIR: FrodoPIR, SimplePIR/DoublePIR
 - Removing the hint again: HintlessPIR, YPIR, Respire.
 - Doubly-efficient PIR: Lin-Mook-Wichs (STOC '23) gets polylog server time with no client state. It is theoretical for now, but it is a major feasibility result ($N^{(1+\epsilon)}$ storage)
- Promising efficiency (computational/communicational/financial)
- Variety of applications

Private Information Retrieval (PIR) *in short*

The trivial solution:

- Client says to server: “I would like to make a query”
- Server sends to client: the whole database (why not?)
- Client looks up the information in the database themselves locally

Pro: very simple (“trivial”) and private!

Con: communication is the size of D (*each record size * number of records*)

Private Information Retrieval (PIR) *in short*

We need practicality:

- We want PIR schemes to *communicate less data* than the whole database, while still not revealing anything about the query; else, it is trivial but infeasible for clients
 - Asking for just half of the database, for example, reveals that the query was in that half even though it is more efficient

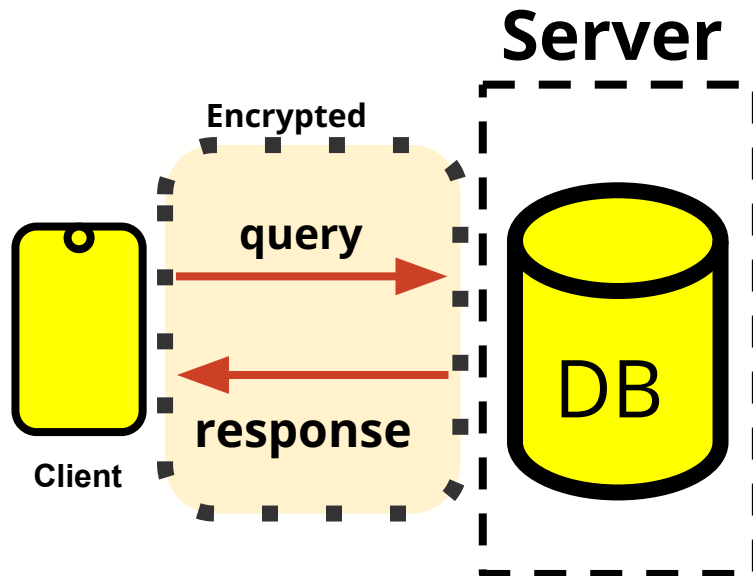
Private Information Retrieval (PIR)

- **Base-line case** for this lecture:

- Single (*semi-honest*) server PIR

- query / response mechanism

- DB.get(i) OR DB.get(kw)
- SELECT * FROM DB WHERE <condition>
- ...



- The database (DB) is assumed public
- May involve offline preprocessing to reduce the costs

Private Information Retrieval (PIR)

Why?

- Multi-server PIR can be more efficient and is built over information-theoretic guarantees, but:
 - No clear process (legal/practical) for finding independent, non-colluding partners:
 - Will they comply with policy regulations?
 - Is there any regional legal problem?
 - Co-deployment seems like a form of collusion
- Single-server efficiency is improving:
 - Allows for small organizations to use it
- We already believe/deploy computational assumptions

Private Information Retrieval (PIR)

Methodology:

Example-driven (case-based) approach: we are going to “learn” PIR by one example: *FrodoPIR* (and in the process *SimplePIR*)

- The lattice part is not so important for our purposes, what matters is the homomorphism

FRODOPIR

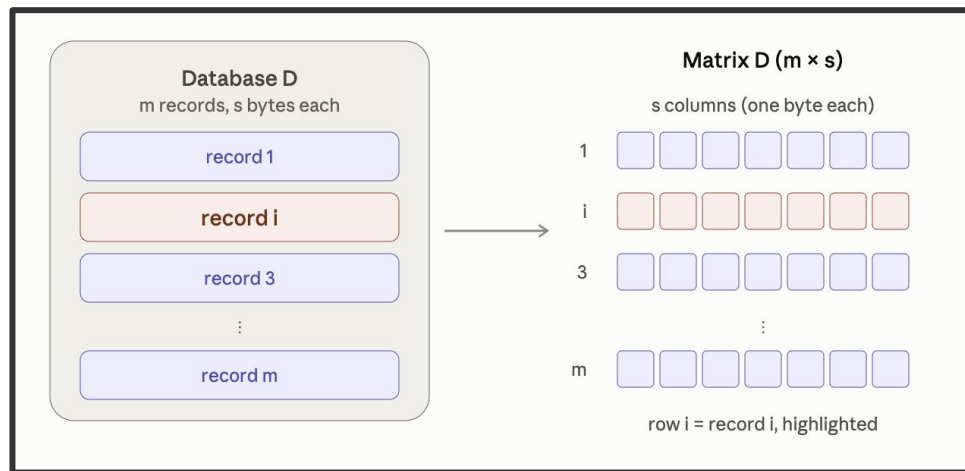
(but also *SimplePIR*)

<https://eprint.iacr.org/2022/981>

<https://eprint.iacr.org/2022/949>

But first...

- Most PIR protocols will model the database D as a matrix
 - For example, a matrix with m rows, each of length s bytes
 - The i th row of the matrix is the i th record of the database



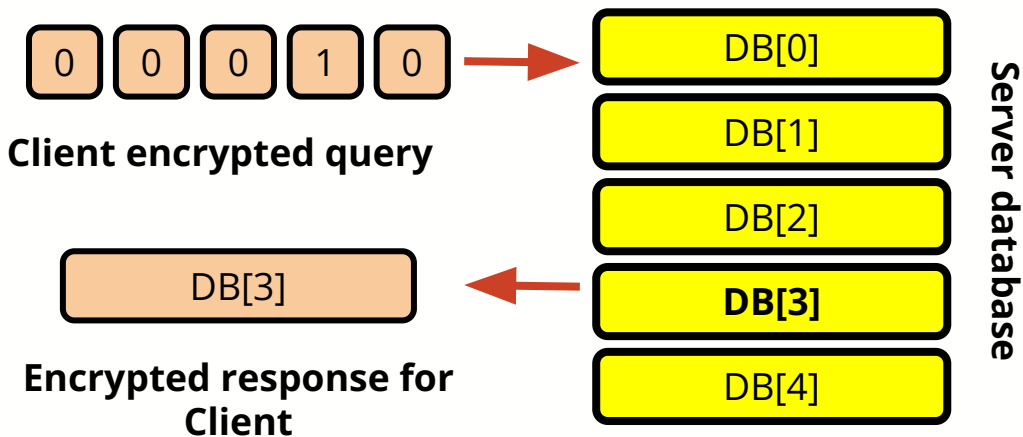
- If you write your query like this: $q = [0\ 0\ 0\ 1\ 0\ 0]$ then what is $q \cdot D$?

Single Server Index-PIR

- The database is indexed by digits (index-based)
- Can be based on **additively homomorphic encryption (AHE)**
- Newer efficient schemes are built directly upon the learning with errors (LWE) problem only (*LWE-based*):
 - **FrodoPIR** [DPC23]
 - SimplePIR [Hen+23]
- Easy to understand from an engineering/implementation perspective
- Highly configurable and practical
- Tailored for efficiency and real-world applications

Single-server Lattice-based PIR

- Based on practical[®] constructions of homomorphic encryption from LWE or RLWE (essentially, lattice-based)
- Better efficiency: stateful model (pre-processing)



Single Server Index-PIR

Based on **additively homomorphic encryption (AHE)**. It is **I-PIR** (index)

$$\text{Client (query): } iv = E(0) \quad E(1) \quad E(2) \quad E(3)$$

$$\text{Server (response): } DB = X(0) \quad X(1) \quad X(2) \quad X(3)$$

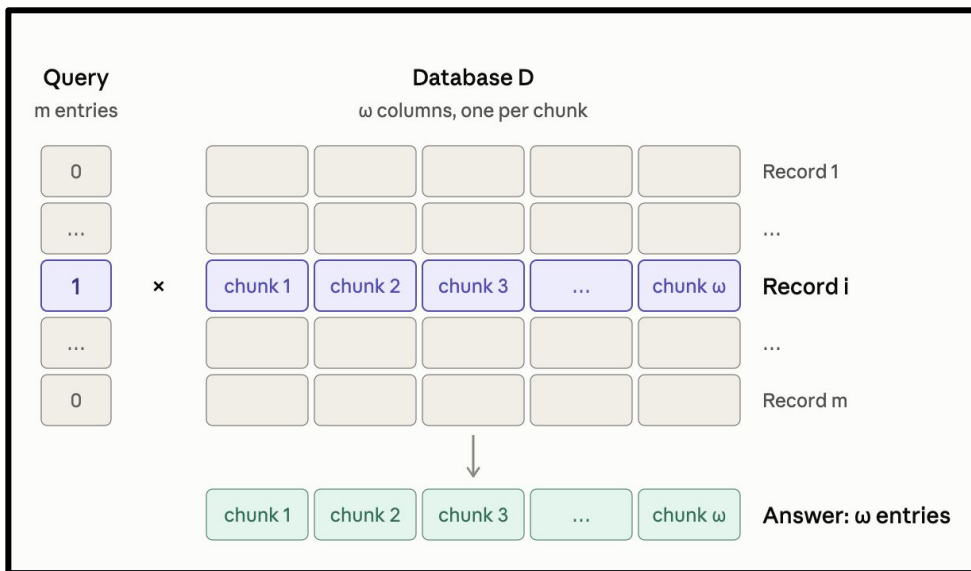
$$\text{Parse: } \langle iv, DB \rangle = \sum_{i=0} E(i \cdot X[i]) = E(X(1))$$

$$E(k, x) + E(k, y) = E(k, x+y)$$

Single-server Lattice-based PIR

- In LWE, we need a small p for correctness, hence, each entry does not completely “fit”:
 - Chunk it: $\omega = \lceil 8s / \log p \rceil$ chunks
 - Set the matrix as $\sqrt{N} \times \sqrt{N}$

FrodoPIR

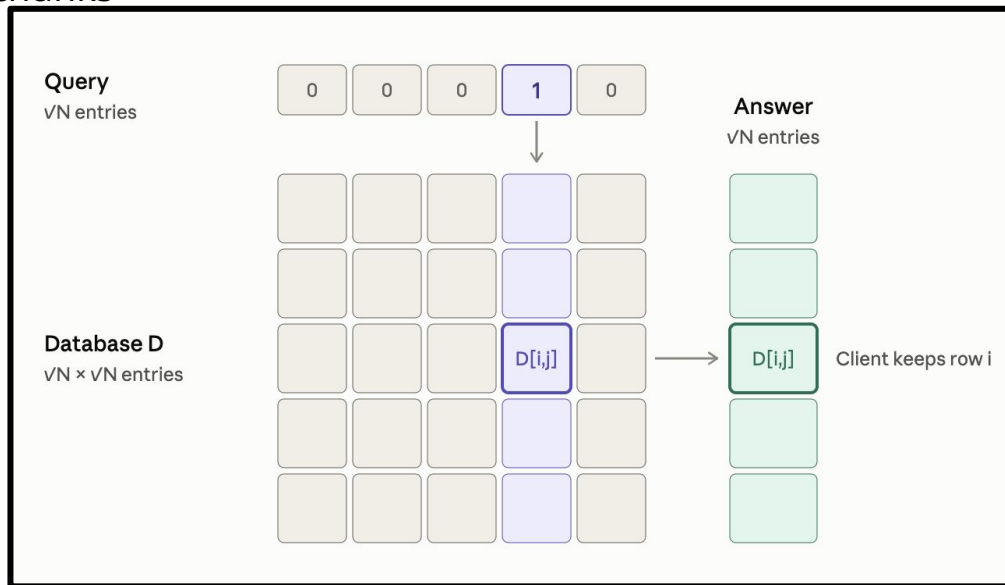


Single-server Lattice-based PIR

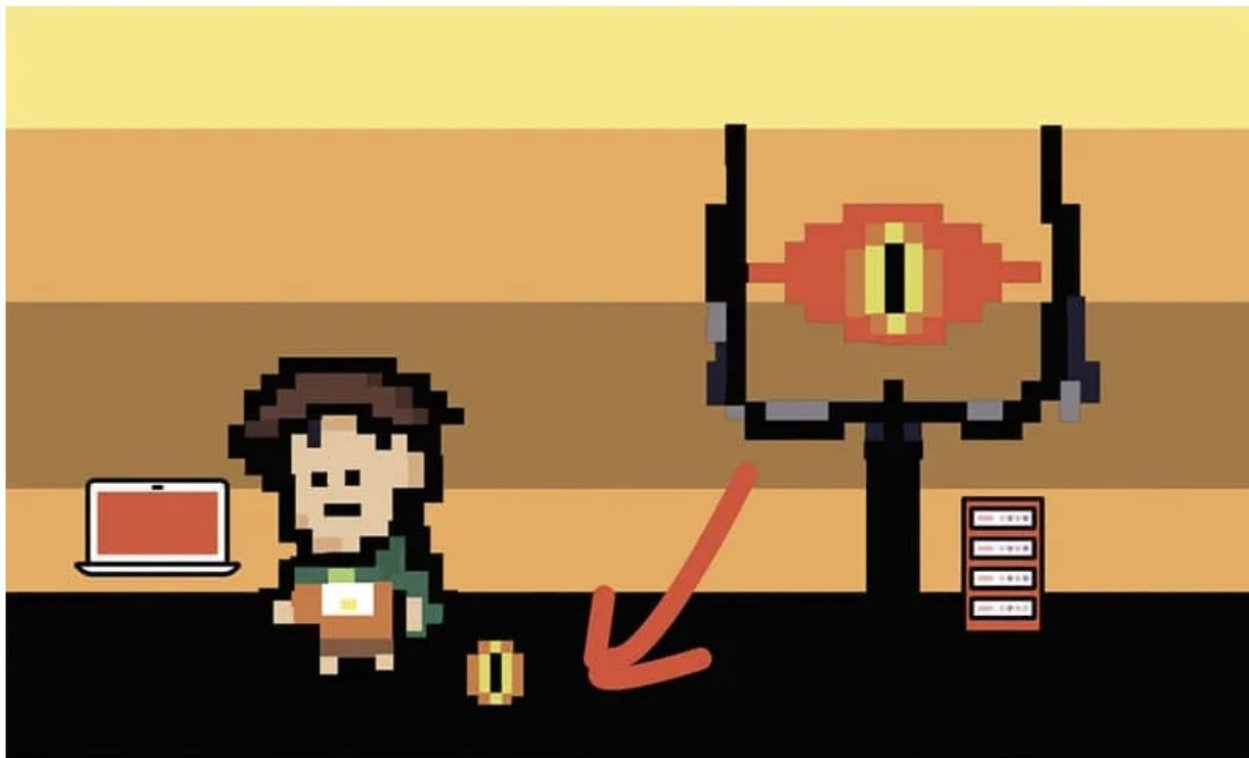
- In LWE, we need a small p for correctness, hence, each entry does not completely “fit”:
 - Chunk it: $\omega = \lceil 8s / \log p \rceil$ chunks
 - Set the matrix as $\sqrt{N} \times \sqrt{N}$

SimplePIR

Client receives more information than needed



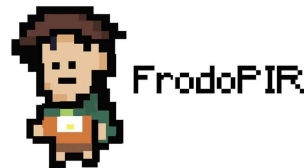
Announcing FrodoPIR!



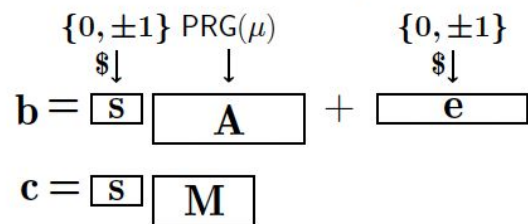
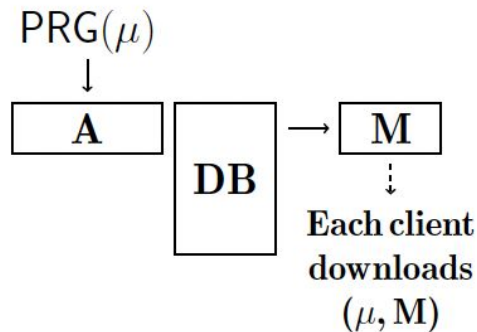
Just as the state of Sauron (its ring) moved to Frodo, we can move the μ and A to the client. The client then can then perform hidden queries to the server, just as Frodo remains hidden from Sauron.

Core ideas

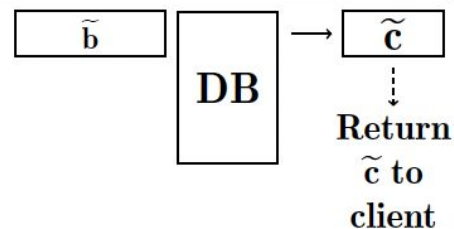
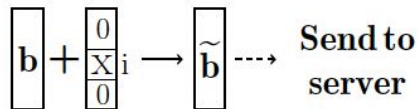
- Built directly upon the learning with errors (LWE) problem *only* (similar to FrodoKEM)
 - Security relies on decisional LWE
 - Security is conservative (128 bits for 2^{52} client queries): some parameters can be modified in order to make the scheme more efficient
- Highly configurable
 - Differences with *SimplePIR*: different pre-processing encoding, and the addition of a query pre-processing stage
- Tailored for efficiency and real-world applications



(1) Server Offline Pre-processing (2) Client Offline Pre-processing



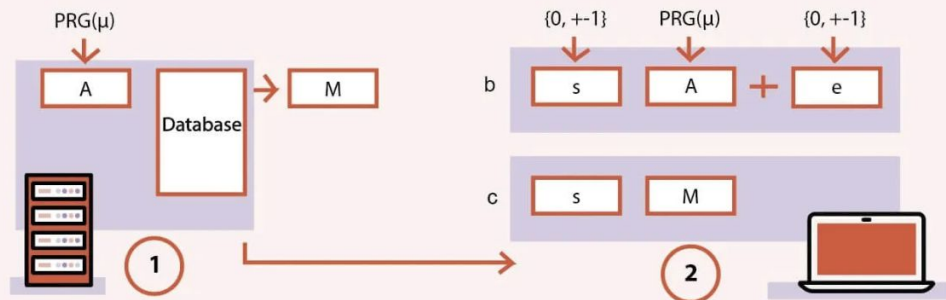
(3) Client Online Query for Index i (4) Server Online Response



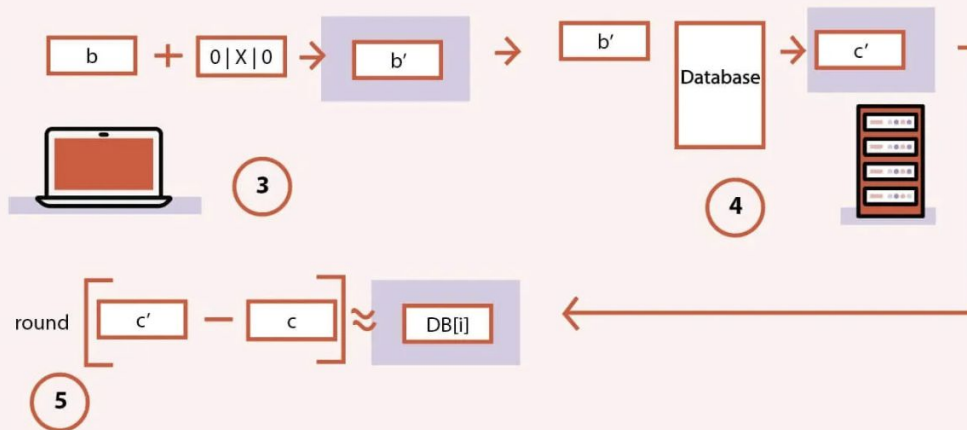
(5) Client Output

$$\text{Round} \left[\begin{bmatrix} \tilde{\mathbf{c}} \end{bmatrix} - \begin{bmatrix} \mathbf{c} \end{bmatrix} \right] \approx \begin{bmatrix} \mathbf{DB}[i] \end{bmatrix}$$

Offline phase



Online phase



Notation

- DB is an array of m elements, each made up of w bits.
- Each entry is associated with the index i that corresponds to its position in the array, and it is a digit.
- There are C clients that will each launch a maximum of c queries against DB.
- LWE:
 - a. n and q are the LWE dimension and modulus, respectively
 - b. ρ is the number of bits packed into each entry of the DB matrix ($0 < \rho < q$)
 - c. χ is the uniform ternary distribution over $\{-1, 0, 1\}$
 - d. λ is the concrete security parameter.
- $\text{PRG}(\mu, n, m, q)$ denotes a pseudorandom generator that expands a seed

$$\mu \in \{0, 1\}^\lambda \in \mathbb{Z}_q^{x \times y}$$

FrodoPIR (offline: server)

- **Server setup:** The server constructs their database containing m elements (as we have seen), and samples a seed $\mu \in \{0, 1\}^\lambda$
- **Server pre-processing:** The server:
 - Derives a matrix $\mathbf{A} \leftarrow \text{PRG}(\mu, n, m, q)$
 - Runs $\mathbf{D} \leftarrow \text{parse}(DB, \rho)$
 - Runs $\mathbf{M} \leftarrow \mathbf{A} \cdot \mathbf{D}$
 - Publishes the pair $(\mu, \mathbf{M}) \in \{0, 1\}^\lambda \times \mathbb{Z}_q^{n \times \omega}$

$$\begin{aligned}\mathbf{A} &\in \mathbb{Z}_q^{n \times m} \\ \mathbf{D} &\in \mathbb{Z}_q^{n \times \omega} \\ \omega &= w / \log(\rho)\end{aligned}$$

The “hint” is $\mathbf{M} \leftarrow \mathbf{A} \cdot \mathbf{D}$

FrodoPIR (offline: server)

- **Setup:** The server constructs their database containing m elements, and samples a seed $\mu \in \{0, 1\}^\lambda$
- **Pre-processing:** The server:
 - Derives a matrix $\mathbf{A} \leftarrow \text{PRG}(\mu, n, m, q)$
 - Runs $\mathbf{D} \leftarrow \text{parse}(DB, \rho)$
 - Runs $\mathbf{M} \leftarrow \mathbf{A} \cdot \mathbf{D}$
 - Publishes the pair $(\mu, \mathbf{M}) \in \{0, 1\}^\lambda \times \mathbb{Z}_q^{n \times \omega}$

$$\begin{aligned}\mathbf{A} &\in \mathbb{Z}_q^{n \times m} \\ \mathbf{D} &\in \mathbb{Z}_q^{n \times \omega} \\ \omega &= w / \log(\rho)\end{aligned}$$

$\mathbf{A}$ remains secure even with multiple queries -2^{52} .

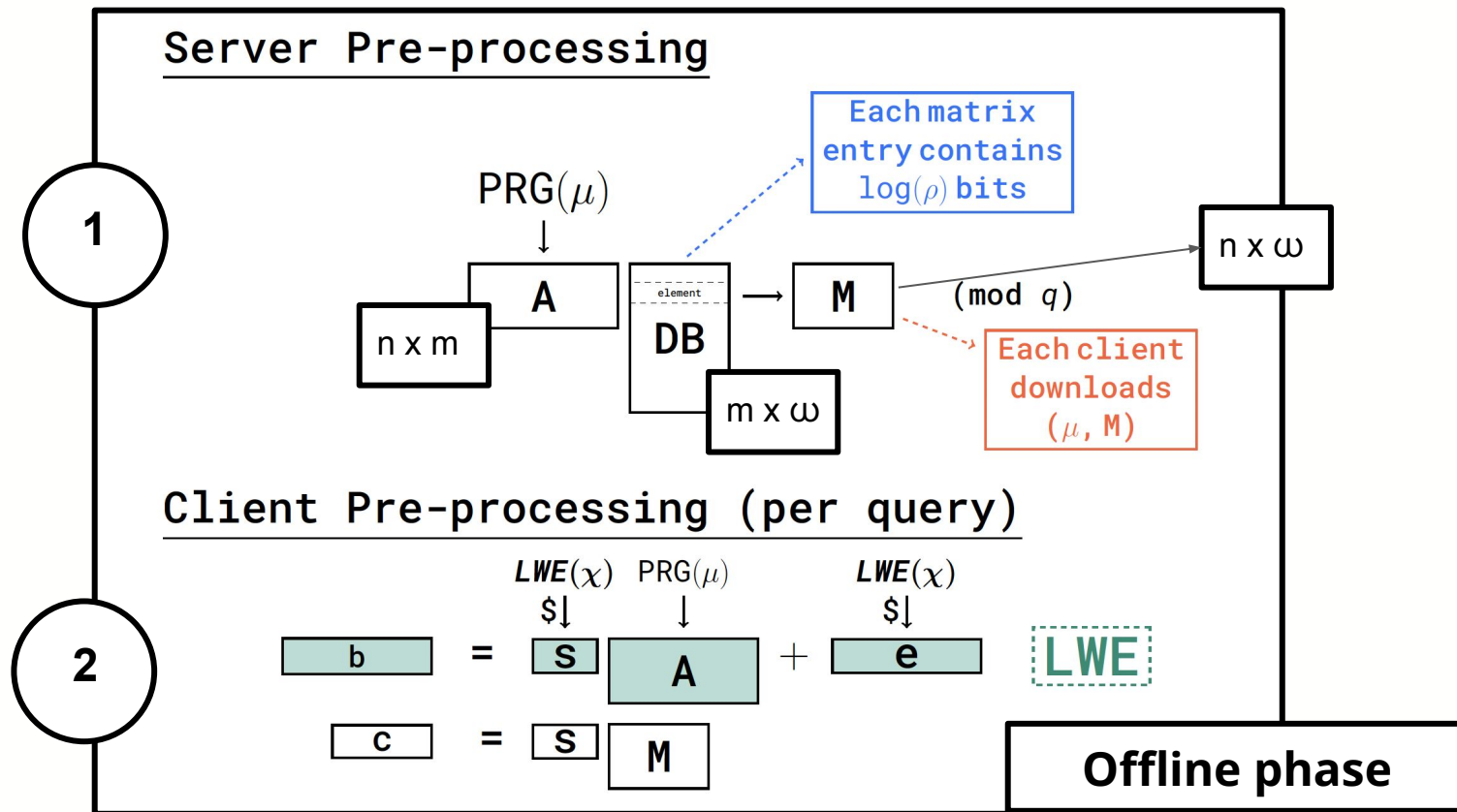
FrodoPIR (offline: client)

Pre-processing. Each client:

- Downloads $(\mu, \mathbf{M})$
- Derives $\mathbf{A} \leftarrow \text{PRG}(\mu, n, m, q)$
- Samples c vectors:
 - $s_j \leftarrow \chi^n \quad e_j \leftarrow \chi^m \quad j \in [c]$
- Computes:
 - $b_j \leftarrow s_j^T \cdot \mathbf{A} + e_j^T \quad \in \mathbb{Z}_q^m$
 - $c_j \leftarrow s_j^T \cdot \mathbf{M} \quad \in \mathbb{Z}_q^\omega$
- Stores the pairs as the set $X = (b_j, c_j)_{j \in [c]}$

Essentially, computes c sets of preprocessed query parameters (optional step).

FrodoPIR (offline)



FrodoPIR (online: client)

Query generation. For the index i that the client wishes to query, the client generates a vector (the all-zero vector except where $f_i[i] = q/\rho$):

$$f_i = (0, \dots, 0, q/\rho, 0, \dots, 0)$$

It then pops a pair (b, c) from internal state and computes:

$$b' = b + f_i$$

The client uses a single set of preprocessed query parameters to produce an “encrypted” query vector, which is sent to the server

FrodoPIR (online: server)

Response. The server receives b' from the client, and responds with:

$$c' \leftarrow b' \cdot \mathbf{D} \in \mathbb{Z}_q^\omega$$

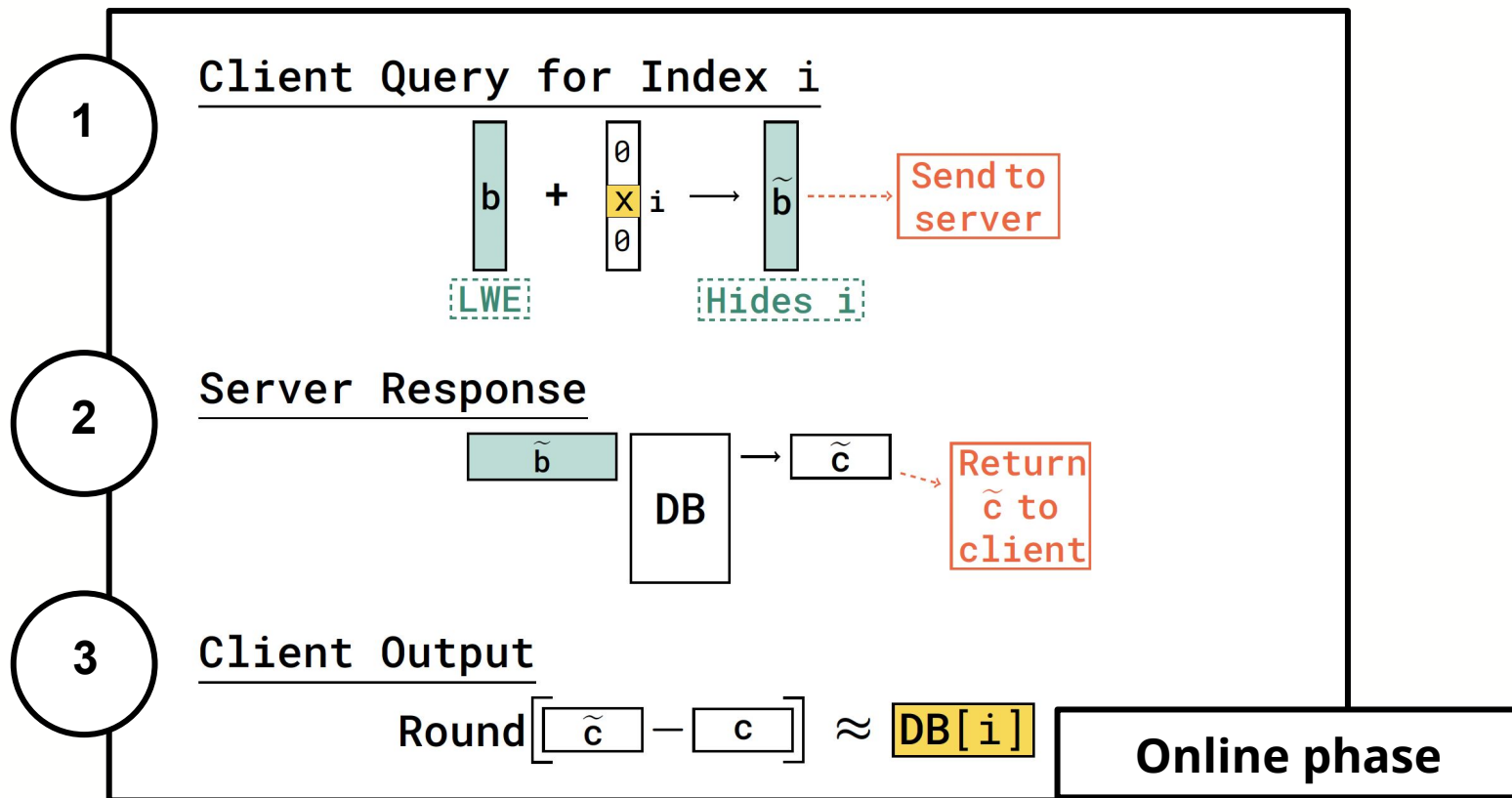
Essentially, the server responds by multiplying the vector with their DB matrix

Post-processing. The client receives c' , and calculates:

$$v \leftarrow \lfloor (c' - c)_\rho \rfloor$$

Essentially, the client get the value by “decrypting” using their pre-processed query parameters)

FrodoPIR (online)



Now, let's discuss

- What is the multiplicative depth of this scheme?
- How much data is sent from the client to the server?
- How much data is sent from the server to the client?

Now, let's discuss

- What is the multiplicative depth of this scheme?
- How much data is sent from the client to the server?
- How much data is sent from the server to the client?

FrodoPIR

- Based on Regev's encryption where the expansion factor is roughly $F = n \approx 1024$, where n is the lattice security parameter
 - Perform bulk of the work in advance and re-use it
 - Additively homomorphic

$$(a, c) = (a, a^T s + e + \lfloor q/\rho \rfloor \cdot \mu) \in \mathbb{Z}_q \times \mathbb{Z}_q$$

- We use a ternary uniform distribution (bounded by $4\sqrt{m}$, with m being the number of samples taken from the distribution)

FrodoPIR Properties

Security: Indistinguishability of client queries. It assumes a semi-honest server that follows the protocol correctly and attempts to learn more based on the client queries they receive:

Server view: b' is distributed uniformly in $\mathbb{Z}_q^m$

under the assumption that decisional-LWE is difficult to solve

- Regev encryption remains secure even when the same matrix A is used to encrypt many messages, provided that each ciphertext uses an independent secret vector s and error vector e

[83] Chris Peikert, Vinod Vaikuntanathan, and Brent Waters. A framework for efficient and composable oblivious transfer.

FrodoPIR Properties

Efficiency. PIR schemes require a communication overhead smaller than the solution of having clients download *the entire server database*. In the stateful PIR case, it should hold when amortizing costs over the number of client queries.

Definition 5. (Efficiency) *For a single client launching c queries, a PIR scheme is efficient if the total client communication overhead is smaller than $|\text{DB}|$.*

Therefore, for stateful schemes, the total client communication cost is calculated using the equation: $\text{comms}(\text{offline}) + c \cdot \text{comms}(\text{online})$.

FrodoPIR Properties

Efficiency.

Definition 5. (Efficiency) *For a single client launching c queries, a PIR scheme is efficient if the total client communication overhead is smaller than $|\text{DB}|$.*

Therefore, for stateful schemes, the total client communication cost is calculated using the equation: $\text{comms}(\text{offline}) + c \cdot \text{comms}(\text{online})$.

	Offline	Online
Client upload	—	$m \log(q)$
Client download	$128 + n\omega \log(q)$	$\omega \log(q)$

FrodoPIR Properties

Efficiency.

	Offline	Online
Client upload	—	$m \log(q)$
Client download	$128 + n\omega \log(q)$	$\omega \log(q)$

$$128 + n\omega \log(q) + c\omega \log(q) < |DB|.$$

Number of DB items ($\log(m)$)		16	17	18	19	20
Offline	Client download (KB)	5682.47	5682.47	5682.47	6313.07	6313.07
	Database preprocessing (s)	92.409	185.30	374.56	825.50	1679.8
	Client derive params (s)	0.5208	1.042	2.1	4.29	8.39
	Client query preprocessing (s)	0.134	0.265	0.532	1.058	2.111
Online	Client query (KB)	256	512	1024	2048	4096
	Server response (KB)	3.203	3.203	3.203	3.556	3.556
	Client query (ms)	0.0177	0.0454	0.0813	0.1565	0.3328
	Server response (ms)	45.74	89.57	179.3	397.06	779.75
	Client output (ms)	0.418	0.4182	0.416	0.4559	0.4627

<https://github.com/brave-experiments/frodo-pir>

Online Performance (ms) of Index-based FrodoPIR [29]				
	DB ($m \times w$)	Query	Response	Parsing
Macbook M1 Max	$2^{16} \times 1024$ B	0.0076956	5.2735	0.18083
	$2^{17} \times 1024$ B	0.017356	10.545	0.18544
	$2^{18} \times 1024$ B	0.055522	21.101	0.18061
	$2^{19} \times 1024$ B	0.1023	47.675	0.20108
	$2^{20} \times 1024$ B	0.21222	100.63	0.20483
EC2 “t2.t2xlarge”	$2^{16} \times 1024$ B	0.11887	29.482	0.34437
	$2^{17} \times 1024$ B	0.080101	50.585	0.34515
	$2^{18} \times 1024$ B	0.20374	118.54	0.3466
	$2^{19} \times 1024$ B	0.48432	263.83	0.3768
	$2^{20} \times 1024$ B	0.85748	537.28	0.37458
EC2 “c5.9xlarge”	$2^{20} \times 256$ B	1.2324	118.46	0.065281
	$2^{17} \times 30$ kB	0.036396	36.396	8.1519
	$2^{14} \times 100$ kB	0.0033412	637.81	26.599

Online Performance (ms) of Index-based FrodoPIR [29]				
	DB ($m \times w$)	Query	Response	Parsing
Macbook M1 Max	$2^{16} \times 1024$ B	0.0076956	5.2735	0.18083
	$2^{17} \times 1024$ B	0.017356	10.545	0.18544
	$2^{18} \times 1024$ B	0.055522	21.101	0.18061
	$2^{19} \times 1024$ B	0.1023	47.675	0.20108
	$2^{20} \times 1024$ B	0.21222	100.63	0.20483
EC2 “t2.t2xlarge”	$2^{16} \times 1024$ B	0.11887	29.482	0.34437
	$2^{17} \times 1024$ B	0.080101	50.585	0.34515
	$2^{18} \times 1024$ B	0.20374	118.54	0.3466
	$2^{19} \times 1024$ B	0.48432	263.83	0.3768
	$2^{20} \times 1024$ B	0.85748	537.28	0.37458
EC2 “c5.9xlarge”	$2^{20} \times 256$ B	1.2324	118.46	0.065281
	$2^{17} \times 30$ kB	0.036396	36.396	8.1519
	$2^{14} \times 100$ kB	0.0033412	637.81	26.599

What are the advantages?

1. It is simple: easy to explain, easy to push to production
2. LWE-based PIR schemes are simple to implement: they require no polynomial arithmetic or fast Fourier transforms
3. LWE-based PIR schemes do not require the server to store any extra per-client state. In contrast, many schemes based on Ring LWE rely on optimizations that require the server to store one “key-switching hint” for each client
4. LWE-based PIR schemes are faster and cheaper: the encryption scheme needs to be linearly (not fully) homomorphic, so we can use smaller and more efficient lattice parameters

But, is this enough?

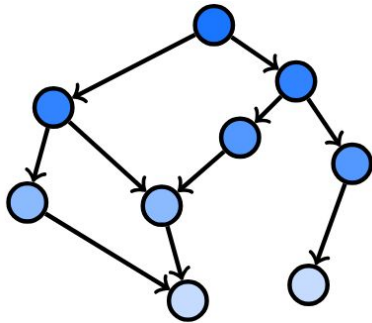
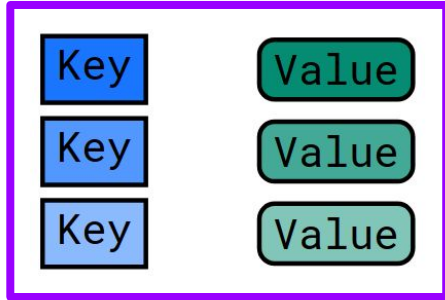
- Databases are not structured in this simple way
 - They are indexed by keywords
 - They are structured as JSON, Graphs, Excel spreadsheets
- The queries we are interested in are not simple:
 - Complex queries with AND/OR statements
 - Combination of database systems
 - Approximate nearest neighbor (ANN) elements
- Databases are constantly updated
- Is the security we assume enough?
 - What about malicious security?
 - What about private databases?

But, is this enough?

- Databases are not structured in this simple way
 - They are indexed by keywords
 - They are structured as JSON, Graphs, Excel spreadsheets
- The queries we are interested in are not simple:
 - Complex queries with AND/OR statements
 - Combination of database systems
 - Approximate nearest neighbor (ANN) elements
- Databases are constantly updated
- Is the security we assume enough?
 - What about malicious security?
 - What about private databases?

**Not all systems are
created equally**

Real databases



col_1	col_2	col_3
x_1	y_1	z_1
x_2	y_2	z_2
x_3	y_3	z_3

Non-uniform data

a	a	b	b
c	c	d	d
e	e	f	f
g	g	h	h

```
{  
  "firstName": "Joe",  
  "lastName": "Jackson",  
  "gender": "male",  
  "age": 28,  
  "address": {  
    "streetAddress": "101",  
    "city": "San Diego",  
    "state": "CA"  
  },  
  "phoneNumbers": [  
    { "type": "home", "number": "7349282382" }  
  ]  
}
```

- ❑ Data-specific privacy?
- ❑ Efficiency for multi-layer keys?
- ❑ Client storage?

a	a	b	b
b	b	b	b
b	c	d	d
d	e	e	f

Goals:

- Design PIR with real databases in mind.

- Security and performance modelling should take **database format** into account.

Which applications?

Some deployments / related technologies exist:

- ❑ Brave ([compromised credential-checking](#), TBD)
- ❑ Blyss (<https://github.com/blyssprivacy/sdk>)
- ❑ Google ([Device Enrollment](#))
- ❑ Microsoft ([Password Monitor](#))

More complex use-cases (not deployed):

- ❑ Approximate nearest-neighbor: [Brave News](#)
- ❑ Private search: [TipToe](#)
- ❑ Oblivious document ranking: [Coeus](#)

Open questions:

- Build complex functions embedded directly into queries
- Basic PIR used as part of higher-level application

Updatable databases

Differing update-cycles depending on application

- ❑ Slower cadence: contact discovery, compromised credentials
- ❑ Faster cadence: safe browsing, recommendation systems (*)

Stateful PIR: require state regeneration with every update

Goals:

- More benchmarking of stateful PIR with incremental updates
- More efficient (and simpler®) stateless PIR

Configurability

Different performance metrics matter to different systems

- ❑ **Financial costs** may be more important than bandwidth for those without hardware
- ❑ **Server load** may be more important for CDNs, Google, etc.
- ❑ **Client load / bandwidth** for mobile devices

Question: Separate approaches for each criteria? Or support for simple re-parametrisation?

Important security properties

- ❑ Does a semi-honest, public DB satisfy all applications?
 - ❑ **Probably not:** compromised credentials, contact-checking...
- ❑ Private DB + semi-honest seems important
 - ❑ Privacy measures are *ad-hoc* (OPRF, masking)
 - ❑ Implications: sub-optimal rounds, not post-quantum...
- ❑ Authenticated/verifiable/malicious PIR exists, is this what we should be using everywhere?

Simplicity®

- ❑ FHE-based PIR is very complex
 - ❑ Libraries are hard to audit/verify
 - ❑ Non-standard security parameters
 - ❑ Low-level optimisations required for PIR
- ❑ AHE-based is simpler and configurable

Question: Do we want **widespread** or **centralised** deployments?

THANK YOU!

@claucece
www.sofiaceli.com
